

CENTRIX ML Threat Intelligence & Architecture

Whitepaper & Technical Rationale: RAG vs. Model Fine-Tuning & Custom Training

1. Executive Summary & Architecture Overview

The CENTRIX Threat Intelligence Engine integrates local machine learning capabilities to enrich security scan findings with deep, actionable vulnerability reasoning. The system combines local Large Language Model (LLM) reasoning via Ollama (qwen2.5-coder) with a high-performance vector store (ChromaDB) to achieve real-time, privacy-preserving threat intelligence without transmitting proprietary codebase findings to external cloud APIs.

Key Architecture Modules:

Multi-Source Advisory Feeds:	Automated sync for NVD (CVE API 2.0), GHSA (GraphQL), CISA-KEV, & Exploit-DB.
Language-Aware Normalization:	Canonical AdvisoryRecord dataclass with language inference (Python, PHP, JS, etc.).
ChromaDB Vector Store:	Dense nomic-embed-text embeddings stored locally with metadata-level language filtering.
Automated Scan Enrichment:	Stage 5 Analyze pipeline automatically enriches findings with structured JSON AI fields.
Anti-Hallucination Guardrails:	Confidence badges (1-10) & automated warnings when cited CVEs miss retrieved context.

2. Comparative Matrix: RAG vs. Fine-Tuning vs. Model Training

A foundational design decision in building CENTRIX ML was evaluating whether to fine-tune an LLM, train a custom model on external security datasets, or implement Retrieval-Augmented Generation (RAG). The table below summarizes key operational trade-offs:

Evaluation Criteria	CENTRIX RAG Pipeline [Chosen]	LLM Fine-Tuning	Full Model Training
Knowledge Recency (New CVEs/Zero-Days)	INSTANT (Real-Time) DB updates in seconds without retraining.	STALE / STATIC Frozen at training epoch. Retraining is expensive.	STALE / STATIC Takes weeks/months to retrain model weights.
Hallucination Risk & Citation Audit	STRICTLY GROUNDED Direct CVE citations & hallucination detection warnings.	HIGH RISK Generates plausible but fake CVE IDs and patch notes.	HIGH RISK Prone to hallucinating parameters & severity scores.
Catastrophic Forgetting	ZERO LOSS Base LLM retains full code parsing & JSON reasoning skills.	SEVERE DEGRADATION Fine-tuning degrades core code syntax & JSON formatting.	HIGH RISK Requires massive multi-task dataset to avoid logic loss.
Data Privacy & Air-Gap Support	100% LOCAL / ON-PREM Vectors stay inside .wraith/chroma_db locally via Ollama.	PRIVACY EXPOSURE Requires exporting logs/findings to training clusters.	HIGH PRIVACY RISK External dataset pipeline exposure risks compliance.
Compute Cost & Infrastructure	ULTRA LOW COST Runs on local developer CPU/GPU. \$0 cluster training cost.	HIGH COST Requires multi-GPU hardware (A100/H100) per update run.	EXTREMELY EXPENSIVE Millions in compute, dataset labeling, and hardware.

3. Deep Technical Rationale: Why RAG Beats Fine-Tuning

A. Real-Time Zero-Day Adaptability vs. Static Training Cutoffs

Threat intelligence is an inherently dynamic domain. Public vulnerability repositories (NVD, GHSA, CISA-KEV) publish over 100 new CVEs and exploit proofs-of-concept every single day. Fine-tuned or custom-trained models suffer from immediate knowledge staleness because their knowledge base is strictly locked at their last training epoch. Retraining a model daily or weekly to incorporate new CVEs requires continuous GPU compute pipelines, dataset curation, and regression testing - making it operationally infeasible.

CENTRIX RAG decouples knowledge storage from reasoning logic. When a new CVE or Exploit-DB entry is ingested into ChromaDB (via background feed sync or `ingest_exploits.py`), it is instantly searchable by the RAG engine within seconds. The underlying LLM (qwen2.5-coder) does not need to be retrained or modified to reason over brand new vulnerabilities.

B. Prevention of Catastrophic Forgetting & Format Instability

Fine-tuning a general-purpose coding LLM on security text datasets triggers 'catastrophic forgetting' - a phenomenon where the model's weight adjustments for vulnerability data degrade its general reasoning capabilities. Specifically, fine-tuned security models frequently lose the ability to parse complex multi-language source code syntax (Python, JS, Java) and fail to output strict JSON schemas required by automated security reporting pipelines.

CENTRIX RAG preserves 100% of the base LLM's coding and instruction-following performance. The model receives raw unmodified source code alongside verbatim retrieved context chunks in the prompt, allowing it to apply pure reasoning without distorting its internal representations.

C. Elimination of Dangerous LLM Hallucinations in Audit Reports

In automated vulnerability assessments (VAPT), hallucinating a non-existent CVE ID or generating invalid exploit syntax destroys report credibility and wastes hundreds of engineering hours on false positives. Fine-tuned models predict text statistically, making them highly prone to inventing realistic-sounding but fake CVE numbers (e.g. 'CVE-2025-99999').

CENTRIX RAG eliminates hallucination risks through a two-tier verification mechanism: 1. Grounded Context Prompting: The LLM is strictly constrained to cite only CVEs present in the retrieved ChromaDB context. 2. Automated Hallucination Detector: `scanner/report_generator.py` inspects LLM citations against retrieved advisories. If the model cites an uncited CVE, CENTRIX injects a prominent '[WARNING] Potential LLM Hallucination' banner in the final report.

Key Takeaway: RAG transforms the LLM from a fallible 'memory bank' into an intelligent, auditable reasoning engine that operates over verified, up-to-date threat intelligence vector stores.

4. Enterprise Privacy, Compliance & Integration Workflow

D. 100% On-Premise Air-Gap Compliance & Privacy

Enterprise vulnerability scanning requires absolute confidentiality. External dataset training or cloud-based fine-tuning requires transmitting internal audit findings, API endpoint signatures, and proprietary source code snippets to external cloud APIs or third-party training clusters - violating compliance mandates (SOC2, ISO27001, HIPAA, GDPR).

CENTRIX ML operates 100% locally. Ollama runs local open-weights models (qwen2.5-coder), nomic-embed-text generates embeddings on-device, and ChromaDB stores vector indices inside .wraith/chroma_db. No data ever leaves the host system.

5. Complete CENTRIX ML Data Flow & Workflow

1. Data Ingestion:	NVD, GHSA, CISA-KEV & Exploit-DB feeds fetch raw advisories into canonical AdvisoryRecords.
2. Local Vector Store:	Text is chunked into 600-char windows, embedded via nomic-embed-text, & indexed in ChromaDB.
3. Scan Execution:	CENTRIX DAST engine probes target endpoints and identifies raw vulnerability findings.
4. RAG Enrichment:	scanner/report_generator.py passes findings to RagEngine for vector retrieval & LLM reasoning.
5. Report Generation:	Enriched findings are output as HTML/PDF reports complete with confidence scores & warnings.
6. REST & CLI Access:	FastAPI endpoints (/api/threat-intel/*) and centrix-intel CLI expose the intelligence pipeline.

6. Conclusion & Recommendation

By choosing Retrieval-Augmented Generation over Model Fine-Tuning and External Dataset Training, CENTRIX delivers an ML-powered vulnerability intelligence engine that is real-time adaptable, mathematically auditable, 100% private, and compute-efficient. This architecture ensures security auditors receive state-of-the-art AI insights backed by verifiable threat advisories without the operational overhead, latency, and hallucination risks of traditional fine-tuning.